My Project

Generated by Doxygen 1.9.8

Chapter 1

My Project

1.1 Documentation

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 4

Class Documentation

4.1 NonlinearShellMCYSE Class Reference

Public Member Functions

- NonlinearShellMCYSE (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &zetGPweigth_in, vector< double > &stresses_in, vector< double > &DMatrix_in)

  Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the stiffness matrix.
  The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

- NonlinearShellMCYSE (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &Displacements_in, vector< double > &DMatrix_in)

  Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the strains and stresses.
  The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

- NonlinearShellMCYSE (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &zetGPweigth_in, vector< double > &Displacements_in, vector< double > &stresses_in, vector< double > &DMatrix_in, vector< double > &stresses_hourglass_in)

  Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the stiffness and/or the internal force vector.
  The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

- size_t **get_Rows** ()
- size_t **get_Cols** ()
- void **get_BMatrix** (vector< double > *BMat_out)
- void **get_BMatrix_Transposed** (vector< double > *BMatT)
- double **get_determinant** ()
- void **Calculate_StiffnessMCYSE** ()

  Calculates the linear stiffness matrix "Stiffness_" of the MCYSE shell element. The calculation of the stiffness matrix was left on a separated method because we might want to instantiate this class for other features than the stiffness matrix and therefore we don't need to waste CPU time of calculating the stiffness matrix if it is not needed.
- vector< double > get_StiffnessMCYSE ()

Get the stiffness matrix "Stiffness_" in vector form. This is a getter only for the stiffness matrix, it does not calculate the stiffness matrix.
The logical steps for the stiffness matrix are: 1. instantiate a new MCYSE class using the appropriate constructor; 2. call the method "Calculate_StiffnessMCYSE()" to calculate the stiffness matrix; 3. call this getter to get the vector "Stiffness_" with the stiffness matrix.

- void set_LocalAxes (vector< double > axesgp_in)

  Set the local axes for the shell MCYSE element. If the local axes are not setted then the API will calculate the local axes.
- vector< double > get_LocalAxes_All ()

  Get the local axes of the shell element.
- void **Calculate_Local_Axes_All_GP** ()
- vector< double > get_StrainsMCYSE ()

  Calculate and get the strain tensor (Strains_) at all Gauss points of the shell element. The strains are defined in the local coordinate system at the Gauss point.
- vector< double > get_StrainsMCYSE_Global ()

  Calculate and get the strain tensor (Strains_Global_) at all Gauss points of the shell element. The strains are defined in the global coordinate system.
- vector< double > get_StressesMCYSE ()

  Calculate and get the elastic stress tensor (stresses_) at all Gauss points of the shell element. The stresses are defined in the local coordinate system at the Gauss point.
- vector< double > get_StressesMCYSE_Global ()

  Calculate and get the stress tensor (stresses_Global) at all Gauss points of the shell element. The stresses are defined in the global coordinate system.
- void **rotate_GP_axes** ()
- vector< double > **Get_Force_Hourglass** ()
- void **Hourglass_Forces_Intermediate** ()
- void **Calculate_Internal_Forces_Explicit** ()

  Calculates the internal force vector "Force_Internal_" for the MCYSE shell element.
- vector< double > Get_Internal_Forces_Explicit ()

  Gets the internal force vector "Force_Internal_" of the MCYSE shell element. The method "Calculate_Internal_↩ Forces_Explicit()" must be used before calling this getter.
- vector< double > **Get_stresses_hourglass** ()

Static Public Attributes

- static string **path** = ""

4.1.1 Constructor & Destructor Documentation

4.1.1.1 NonlinearShellMCYSE() [1/3]

```
NonlinearShellMCYSE::NonlinearShellMCYSE (
            string path_in,
            vector< vector< double > > & xyz_in,
            vector< vector< double > > & vnor_in,
            vector< double > & thick_in,
            vector< vector< double > > & xlocal_in,
            vector< vector< double > > & ylocal_in,
            vector< double > & zetGPcoord_in,
            vector< double > & zetGPweigth_in,
            vector< double > & stresses_in,
            vector< double > & DMatrix_in )
```

Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the stiffness matrix.
The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: vec1 = {0.0, 1.0, 0.0};

2. define "xloc" as the result of the following cross-product: xloc = vec1 x V3 (V3 is the normal vector at the node);

3. if (xloc.empty()) { xloc = {1.0, 0.0, 0.0} };

4. V1 = xloc;

5. V2 = V3 x V1.

Parameters

path_in	String with the full path to the license file.
xyz_in	Nodal coordinates of the four shell element nodes, stored in xyz_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
vnor_in	Normal vector of the four shell element nodes, stored in vnor_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.
thick_in	Element's thickness at the nodes, stored in thick_in[0:3], where each index is a node of the MCYSE shell element.
xlocal_in	V1 local nodal vector stored in xlocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in), V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
ylocal_in	V2 local nodal vector stored in ylocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in), V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
zetGPcoord_in	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
zetGPweigth↩ _in	Weights of the Gauss integration points along the thickness direction of the shell. The size of this vector must be the same as the size of "zetGPcoord_in".
stresses_in	The stresses at the integration points must be provided as input to the class constructor. These stresses are required for the computation of the internal force vector and the initial stress stiffness matrix. The stress input format is defined as follows: Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point along the thickness direction. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0. For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.0), the ordering in stresses_ is: [Sxx^zet1, Syy^zet1, Sxy^zet1, Sxz^zet1, Syz^zet1, Sxx^zet2, Syy^zet2, Sxy^zet2, Sxz^zet2, Syz^zet2]

Parameters

DMatrix_in	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points along the thickness direction of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { 2. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector: 2.1 for (int i = 0; i < 25; ++i) { DMatrix_all.push_back(DMatrix[i]); }

4.1.1.2 NonlinearShellMCYSE() [2/3]

```
NonlinearShellMCYSE::NonlinearShellMCYSE (
            string path_in,
            vector< vector< double > > & xyz_in,
            vector< vector< double > > & vnor_in,
            vector< double > & thick_in,
            vector< vector< double > > & xlocal_in,
            vector< vector< double > > & ylocal_in,
            vector< double > & zetGPcoord_in,
            vector< double > & Displacements_in,
            vector< double > & DMatrix_in )
```

Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the strains and stresses. The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: vec1 = {0.0, 1.0, 0.0};
2. define "xloc" as the result of the following cross-product: xloc = vec1 x V3 (V3 is the normal vector at the node);
3. if (xloc.empty()) { xloc = {1.0, 0.0, 0.0} };
4. V1 = xloc;
5. V2 = V3 x V1.

Parameters

path_in	String with the full path to the license file.
xyz_in	Nodal coordinates of the four shell element nodes, stored in xyz_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
vnor_in	Normal vector of the four shell element nodes, stored in vnor_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.

Parameters

thick_in	Element's thickness at the nodes, stored in thick_in[0:3], where each index is a node of the MCYSE shell element.
xlocal_in	V1 local nodal vector stored in xlocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in), V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
ylocal_in	V2 local nodal vector stored in ylocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in), V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
zetGPcoord_in	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
Displacements_in	Displacement vector for the element. The shell MCYSE has 4 nodes and each node has 6 degrees of freedom. The size of this vector is 4x6 = 24.
DMatrix_in	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points along the thickness direction of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { 2. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector: 2.1 for (int i = 0; i < 25; ++i) { DMatrix_all.push_back(DMatrix[i]); }

4.1.1.3 NonlinearShellMCYSE() [3/3]

```
NonlinearShellMCYSE::NonlinearShellMCYSE (
            string path_in,
            vector< vector< double > > & xyz_in,
            vector< vector< double > > & vnor_in,
            vector< double > & thick_in,
            vector< vector< double > > & xlocal_in,
            vector< vector< double > > & ylocal_in,
            vector< double > & zetGPcoord_in,
            vector< double > & zetGPweigth_in,
            vector< double > & Displacements_in,
            vector< double > & stresses_in,
            vector< double > & DMatrix_in,
            vector< double > & stresses_hourglass_in )
```

Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the stiffness and/or the internal force vector.
The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: vec1 = {0.0, 1.0, 0.0};

2. define "xloc" as the result of the following cross-product: xloc = vec1 x V3 (V3 is the normal vector at the node);

3. if (xloc.empty()) { xloc = {1.0, 0.0, 0.0} };

4. V1 = xloc;

5. V2 = V3 x V1.

Parameters

path_in	String with the full path to the license file.
xyz_in	Nodal coordinates of the four shell element nodes, stored in xyz_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
vnor_in	Normal vector of the four shell element nodes, stored in vnor_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.
thick_in	Element's thickness at the nodes, stored in thick_in[0:3], where each index is a node of the MCYSE shell element.
xlocal_in	V1 local nodal vector stored in xlocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in), V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
ylocal_in	V2 local nodal vector stored in ylocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in), V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
zetGPcoord_in	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
zetGPweigth_in	Weights of the Gauss integration points along the thickness direction of the shell. The size of this vector must be the same as the size of "zetGPcoord_in".
Displacements_in	Displacement vector for the element. The shell MCYSE has 4 nodes and each node has 6 degrees of freedom. The size of this vector is 4x6 = 24.
stresses_in	The stresses at the integration points must be provided as input to the class constructor. These stresses are required for the computation of the internal force vector and the initial stress stiffness matrix. The stress input format is defined as follows: Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point along the thickness direction. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0. For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.0), the ordering in stresses_ is: [Sxx^zet1, Syy^zet1, Sxy^zet1, Sxz^zet1, Syz^zet1, Sxx^zet2, Syy^zet2, Sxy^zet2, Sxz^zet2, Syz^zet2]

Parameters

DMatrix_in	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points along the thickness direction of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { 2. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector: 2.1 for (int i = 0; i < 25; ++i) { DMatrix_all.push_back(DMatrix[i]); }
stresses_hourglass_in	Hourglass stresses updated from the incremental displacements (defined in "Displacements_in"). At the input for this constructor "stresses_hourglass_in" are the hourglass stresses at configuration "n". In the method "Calculate_Internal_Forces_Explicit()", the incremental hourglass stresses are calculated and the "stresses_hourglass_" vector is updated so that the getter "Get_stresses_hourglass()" can return the hourglass stresses at configuration "n+1".

4.1.2 Member Function Documentation

4.1.2.1 Get_Internal_Forces_Explicit()

```
vector< double > NonlinearShellMCYSE::Get_Internal_Forces_Explicit ( )  [inline]
```

Gets the internal force vector "Force_Internal_" of the MCYSE shell element. The method "Calculate_Internal_Forces_Explicit()" must be used before calling this getter.

Returns

vector<double> This return is the internal force vector "Force_Internal_" with size of 24 = 4 nodes x 6 dof per node (the DOF number 6 is associated with the drill DOF which is always = 0 in this element formulation).

4.1.2.2 get_LocalAxes_All()

```
vector< double > NonlinearShellMCYSE::get_LocalAxes_All ( )  [inline]
```

Get the local axes of the shell element.

Returns

vector<double>

4.1.2.3 get_StiffnessMCYSE()

```
vector< double > NonlinearShellMCYSE::get_StiffnessMCYSE ( )
```

Get the stiffness matrix "Stiffness_" in vector form. This is a getter only for the stiffness matrix, it does not calculate the stiffness matrix.
The logical steps for the stiffness matrix are: 1. instantiate a new MCYSE class using the appropriate constructor; 2. call the method "Calculate_StiffnessMCYSE()" to calculate the stiffness matrix; 3. call this getter to get the vector "Stiffness_" with the stiffness matrix.

Returns

vector<double> The return is the stiffness matrix "Stiffness_" in vector form (of size = 576). The first elements in the output vector (Stiffness_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 stiffness (stif[0][0:23]), the next 24 elements in the output vector (Stiffness_[24:47]) correspond to stif[1][0:23], and so on.

4.1.2.4 get_StrainsMCYSE()

```
vector< double > NonlinearShellMCYSE::get_StrainsMCYSE ( )
```

Calculate and get the strain tensor (Strains_) at all Gauss points of the shell element. The strains are defined in the local coordinate system at the Gauss point.

Returns

vector<double> The strains input format is defined as follows:
Exx, Eyy, Exy, Exz, Eyz, i.e., five strain components per integration point along the thickness direction. The strains are stored in the "Strains_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.↩
0), the ordering in "Strains_" is:
[Exx^zet1, Eyy^zet1, Exy^zet1, Exz^zet1, Eyz^zet1, Exx^zet2, Eyy^zet2, Exy^zet2, Exz^zet2, Eyz^zet2]

4.1.2.5 get_StrainsMCYSE_Global()

```
vector< double > NonlinearShellMCYSE::get_StrainsMCYSE_Global ( )
```

Calculate and get the strain tensor (Strains_Global_) at all Gauss points of the shell element. The strains are defined in the global coordinate system.

Returns

vector<double> The global strains input format is defined as follows:
EGxx, EGyy, EGxy, EGxz, EGyz, i.e., five strain components per integration point along the thickness direction. The global strains are stored in the "Strains_Global_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.0), the ordering in "Strains_Global_" is:
[EGxx^zet1, EGyy^zet1, EGxy^zet1, EGxz^zet1, EGyz^zet1, EGxx^zet2, EGyy^zet2, EGxy^zet2, EGxz^zet2, EGyz^zet2]

4.1.2.6 get_StressesMCYSE()

```
vector< double > NonlinearShellMCYSE::get_StressesMCYSE ( )
```

Calculate and get the elastic stress tensor (stresses_) at all Gauss points of the shell element. The stresses are defined in the local coordinate system at the Gauss point.

Returns

vector<double> The stress input format is defined as follows:
Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point along the thickness direction. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.↩0), the ordering in stresses_ is:
[Sxx^zet1, Syy^zet1, Sxy^zet1, Sxz^zet1, Syz^zet1, Sxx^zet2, Syy^zet2, Sxy^zet2, Sxz^zet2, Syz^zet2].

4.1.2.7 get_StressesMCYSE_Global()

```
vector< double > NonlinearShellMCYSE::get_StressesMCYSE_Global ( )
```

Calculate and get the stress tensor (stresses_Global) at all Gauss points of the shell element. The stresses are defined in the global coordinate system.

Returns

vector<double> The global stress input format is defined as follows:
SGxx, SGyy, SGxy, SGxz, SGyz, i.e., five global stress components per integration point along the thickness direction. The global stresses are stored in the "stresses_Global" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.0), the ordering in "stresses_Global" is:
[SGxx^zet1, SGyy^zet1, SGxy^zet1, SGxz^zet1, SGyz^zet1, SGxx^zet2, SGyy^zet2, SGxy^zet2, SGxz^zet2, SGyz^zet2].

4.1.2.8 set_LocalAxes()

```
void NonlinearShellMCYSE::set_LocalAxes (
            vector< double > axesgp_in )  [inline]
```

Set the local axes for the shell MCYSE element. If the local axes are not setted then the API will calculate the local axes.

Parameters

axesgp↩ _in	

The documentation for this class was generated from the following file:

- /home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearShellMCYSE/NonlinearShellMCYSE.h

4.2 NonlinearShellMITC4 Class Reference

Public Member Functions

- NonlinearShellMITC4 (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &zetGPweigth_in, vector< double > &stresses_in, vector< double > &DMatrix_in)

  Constructor for a new Nonlinear Shell MITC4 object. This constructor can be used for the stiffness matrix and for the internal force vector.
  The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

- NonlinearShellMITC4 (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &Displacements_in, vector< double > &DMatrix_in)

  Constructor for a new Nonlinear Shell MITC4 object. This constructor can be used for the strains and stresses.
  The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

- size_t **get_Rows** ()
- size_t **get_Cols** ()
- void get_BMatrix (int igaus_in, int izet_in, double zet_in, vector< double > *BMat_out)

  *Calculates the Strain-Displacement matrix (B-matrix) at the Gauss point defined by "izet_in" and "igaus_in". *** Strains = B-matrix . displ ***.*
- void get_BMatrix_Transposed (vector< double > *BMatT)

  Get the Strain-Displacement matrix transposed (B-matrix)^T at the Gauss point defined by "izet_in" and "igaus_in". warning: the method "get_BMatrix" must be called first to construct the B-Matrix at the corresponding Gauss point. Only then, this method can be called for the transposed Strain-Displacement matrix.
- void set_LocalAxes_All (vector< double > axesgp_in)

  Set the local axes at the Gauss integration points of the element. If the local axes are not setted then the API will calculate the local axes at the GPs. If the MITC4 shell is to be used for composites or planar plastic anisotropy then the local axes at the GPs should be setted after the instantiation of the shell MITC4 class.
- vector< double > get_LocalAxes_All ()

  Get the local axes at all integration points of the shell element. The return is a vector with the axes at all GPs which are stored in the same order as explained in the setter - "set_LocalAxes_All".
- void **Calculate_Local_Axes_All_GP** ()

  calculates the local axes at each in-plane Gauss point (located at the mid-surface (zet = 0.0) of the shell). The axes at zet = 0.0 are then copied to all other Gauss points of the MITC4 shell element.
- vector< double > Calculate_StiffnessMITC4 ()

  Calculates the linear stiffness matrix "Stiffness_" of the MITC4 shell element. The calculation of the stiffness matrix was left on a separated method because we might want to instantiate this class for other features than the stiffness matrix and therefore we don't need to waste CPU time of calculating the stiffness matrix if it is not needed.
- vector< double > get_StiffnessMITC4 ()

  Get the stiffness matrix "Stiffness_" in vector form. This is a getter only for the stiffness matrix, it does not calculate the stiffness matrix.
- vector< double > get_Stiffness_InitialStress ()

  Calculates the Initial Stress Stiffness matrix (or geometric stiffness) for the MITC4 shell element. This stiffness is fundamental for nonlinear continuum analysis.
- vector< double > Calculate_Internal_Forces ()

  Calculates the Internal Force vector "Internal_Force_" for the MITC4 shell element. The stresses must be passed first to the instance of this element through its constructor. Note that the right constructor must be used.
- vector< double > get_Internal_Forces ()

  Getter for the Internal Force vector "Internal_Force_".
- void **Calculate_Stiffness_Internal** ()

Compute the total stiffness matrix and the internal force vector. The total stiffness is the sum of the linear stiffness (Stiffness_) and the initial-stress stiffness (StiffnessInitialStress_).
From a CPU-efficiency perspective, in nonlinear continuum mechanics it is advantageous to compute the stiffness matrix and the internal force vector simultaneously.
After calling this method, the method "get_StiffnessMITC4()" must be called for getting the total stiffness, which is stored in the vector "Stiffness_", and the method "get_Internal_Forces()" for getting the internal force vector.

- vector< double > get_StrainsMITC4 ()

  Calculate and get the strain tensor (Strains_) at all Gauss points of the shell element. The strains are defined in the local coordinate system at the Gauss point.
- vector< double > get_StrainsMITC4_Global ()

  Calculate and get the strain tensor (Strains_Global_) at all Gauss points of the shell element. The strains are defined in the global coordinate system.
- void **rotate_GP_axes** ()
- vector< double > get_StressesMITC4 ()

  Calculate and get the stress tensor (stresses_) at all Gauss points of the shell element. The stresses are defined in the local coordinate system at the Gauss point.
- vector< double > get_StressesMITC4_Global ()

  Calculate and get the stress tensor (stresses_Global) at all Gauss points of the shell element. The stresses are defined in the global coordinate system.
- void **Jacobi** (vector< vector< double > > &ain, vector< vector< double > > &bin, vector< double > &eigenvalues_, vector< vector< double > > &eigenvectors_)

Static Public Attributes

- static string **path** = ""

4.2.1 Constructor & Destructor Documentation

4.2.1.1 NonlinearShellMITC4() [1/2]

```
NonlinearShellMITC4::NonlinearShellMITC4 (
            string path_in,
            vector< vector< double > > & xyz_in,
            vector< vector< double > > & vnor_in,
            vector< double > & thick_in,
            vector< vector< double > > & xlocal_in,
            vector< vector< double > > & ylocal_in,
            vector< double > & zetGPcoord_in,
            vector< double > & zetGPweigth_in,
            vector< double > & stresses_in,
            vector< double > & DMatrix_in )
```

Constructor for a new Nonlinear Shell MITC4 object. This constructor can be used for the stiffness matrix and for the internal force vector.
The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: vec1 = {0.0, 1.0, 0.0};

2. define "xloc" as the result of the following cross-product: xloc = vec1 x V3 (V3 is the normal vector at the node);

3. if (xloc.empty()) { xloc = {1.0, 0.0, 0.0} };

4. V1 = xloc;

5. V2 = V3 x V1.

Parameters

path_in	String with the full path to the license file.
xyz_in	Nodal coordinates of the four shell element nodes, stored in xyz_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
vnor_in	Normal vector of the four shell element nodes, stored in vnor_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.
thick_in	Element's thickness at the nodes, stored in thick_in[0:3], where each index is a node of the MITC4 shell element.
xlocal_in	V1 local nodal vector stored in xlocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in),V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
ylocal_in	V2 local nodal vector stored in ylocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in),V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
zetGPcoord_in	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
zetGPweigth↩ _in	Weights of the Gauss integration points along the thickness direction of the shell. The size of this vector must be the same as the size of "zetGPcoord_in".
stresses_in	The stresses at the integration points must be provided as input to the class constructor. These stresses are required for the computation of the internal force vector and the initial stress stiffness matrix. The stress input format is defined as follows: Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0. For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.0), the ordering in stresses_ is: [Sxx_I^zet1, Syy_I^zet1, Sxy_I^zet1, Sxz_I^zet1, Syz_I^zet1,...,Sxx_IV^zet1, Syy_IV^zet1, Sxy_IV^zet1, Sxz_IV^zet1, Syz_IV^zet1, Sxx_I^zet2, Syy_I^zet2, Sxy_I^zet2, Sxz_I^zet2, Syz_I^zet2,...,Sxx_IV^zet2, Syy_IV^zet2, Sxy_IV^zet2, Sxz_IV^zet2, Syz_IV^zet2]

Parameters

DMatrix_in	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points of the shell element. The procedure to construct this vector is the following:
	1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) {
	2. Loop in the 4 in-plane GPs: for (int ig = 0; ig < 4; ++ig) {
	3. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector:
	3.1 for (int i = 0; i < 25; ++i) { DMatrix_all.push_back(DMatrix[i]); }

4.2.1.2 NonlinearShellMITC4() [2/2]

```
NonlinearShellMITC4::NonlinearShellMITC4 (
            string path_in,
            vector< vector< double > > & xyz_in,
            vector< vector< double > > & vnor_in,
            vector< double > & thick_in,
            vector< vector< double > > & xlocal_in,
            vector< vector< double > > & ylocal_in,
            vector< double > & zetGPcoord_in,
            vector< double > & Displacements_in,
            vector< double > & DMatrix_in )
```

Constructor for a new Nonlinear Shell MITC4 object. This constructor can be used for the strains and stresses. The procedure for calculculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: vec1 = {0.0, 1.0, 0.0};
2. define "xloc" as the result of the following cross-product: xloc = vec1 x V3 (V3 is the normal vector at the node);
3. if (xloc.empty()) { xloc = {1.0, 0.0, 0.0} };
4. V1 = xloc;
5. V2 = V3 x V1.

Parameters

path_in	String with the full path to the license file.
xyz_in	Nodal coordinates of the four shell element nodes, stored in xyz_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.

Parameters

vnor_in	Normal vector of the four shell element nodes, stored in vnor_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.
thick_in	Element's thickness at the nodes, stored in thick_in[0:3], where each index is a node of the MITC4 shell element.
xlocal_in	V1 local nodal vector stored in xlocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in),V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
ylocal_in	V2 local nodal vector stored in ylocal_in[0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in),V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
zetGPcoord_in	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
Displacements_in	Displacement vector for the element. The shell MITC4 has 4 nodes and each node has 6 degrees of freedom. The size of this vector is 4x6 = 24.

DMatrix_in	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points of the shell element. The procedure to construct this vector is the following:
	1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) {
	2. Loop in the 4 in-plane GPs: for (int ig = 0; ig < 4; ++ig) {
	3. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector:
	3.1 for (int i = 0; i < 25; ++i) { DMatrix_all.push_back(DMatrix[i]); }

4.2.2 Member Function Documentation

4.2.2.1 Calculate_Internal_Forces()

```
vector< double > NonlinearShellMITC4::Calculate_Internal_Forces ( )
```

Calculates the Internal Force vector "Internal_Force_" for the MITC4 shell element. The stresses must be passed first to the instance of this element through its constructor. Note that the right constructor must be used.

Returns

vector<double> The return is the Internal Force vector of size = 24.

4.2.2.2 Calculate_StiffnessMITC4()

```
vector< double > NonlinearShellMITC4::Calculate_StiffnessMITC4 ( )
```

Calculates the linear stiffness matrix "Stiffness_" of the MITC4 shell element. The calculation of the stiffness matrix was left on a separated method because we might want to instantiate this class for other features than the stiffness matrix and therefore we don't need to waste CPU time of calculating the stiffness matrix if it is not needed.

Returns

vector<double> The return is the stiffness matrix "Stiffness_" in vector form (of size = 576). The first elements in the output vector (Stiffness_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 stiffness (stif[0][0:23]), the next 24 elements in the output vector (Stiffness_[24:47]) correspond to stif[1][0:23], and so on.

4.2.2.3 get_BMatrix()

```
void NonlinearShellMITC4::get_BMatrix (
            int igaus_in,
            int izet_in,
            double zet_in,
            vector< double > * BMat_out )
```

Calculates the Strain-Displacement matrix (B-matrix) at the Gauss point defined by "izet_in" and "igaus_in". *** Strains = B-matrix . displ ***.

Parameters

igaus_in	Number of the in-plane Gauss point (0:3).
izet_in	Number of the integration point along the thickness direction.
zet_in	Natural coordinate of the integration point along the thickness direction.
BMat_out	Strain-Displacement matrix of the MITC4 shell element in vector form and defined in the local GP coordinate system (output). This vector is organised as follows: BMat_out[0:23] corresponds to "Strains_xx"; BMat_out[24:47] corresponds to "Strains_yy"; BMat_out[48:71] corresponds to "Strains_xy"; BMat_out[72:95] corresponds to "Strains_xz"; BMat_out[96:119] corresponds to "Strains_yz";

4.2.2.4 get_BMatrix_Transposed()

```
void NonlinearShellMITC4::get_BMatrix_Transposed (
            vector< double > * BMatT )  [inline]
```

Get the Strain-Displacement matrix transposed (B-matrix)^T at the Gauss point defined by "izet_in" and "igaus_in". warning: the method "get_BMatrix" must be called first to construct the B-Matrix at the corresponding Gauss point. Only then, this method can be called for the transposed Strain-Displacement matrix.

Parameters

BMatT	Transposed Strain-Displacement matrix stored in vector form (output).

4.2.2.5 get_Internal_Forces()

```
vector< double > NonlinearShellMITC4::get_Internal_Forces ( )  [inline]
```

Getter for the Internal Force vector "Internal_Force_".

Returns

vector<double> The return is the Internal Force vector "Internal_Force_" of size = 24.

4.2.2.6 get_LocalAxes_All()

```
vector< double > NonlinearShellMITC4::get_LocalAxes_All ( )  [inline]
```

Get the local axes at all integration points of the shell element. The return is a vector with the axes at all GPs which are stored in the same order as explained in the setter - "set_LocalAxes_All".

Returns

vector<double> The structure of the return vector "axesgp_all_" is the same as the structure of the "axesgp↩ _in" vector defined in the setter "set_LocalAxes_All(vector<double> axesgp_in) { axesgp_all_ = axesgp_in; }"

4.2.2.7 get_Stiffness_InitialStress()

```
vector< double > NonlinearShellMITC4::get_Stiffness_InitialStress ( )
```

Calculates the Initial Stress Stiffness matrix (or geometric stiffness) for the MITC4 shell element. This stiffness is fundamental for nonlinear continuum analysis.

Returns

vector<double> The return is the Initial Stress Stiffness matrix, "StiffnessInitialStress_", stored in vector form (of size = 576). The first elements in the output vector (StiffnessInitialStress_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 "StifInitialStress"
(StifInitialStress[0][0:23]), the next 24 elements in the output vector (StiffnessInitialStress_[24:47]) correspond to StifInitialStress[1][0:23], and so on.

4.2.2.8 get_StiffnessMITC4()

```
vector< double > NonlinearShellMITC4::get_StiffnessMITC4 ( ) [inline]
```

Get the stiffness matrix "Stiffness_" in vector form. This is a getter only for the stiffness matrix, it does not calculate the stiffness matrix.

Returns

vector<double> The return is the stiffness matrix "Stiffness_" in vector form (of size = 576). The first elements in the output vector (Stiffness_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 stiffness (stif[0][0:23]), the next 24 elements in the output vector (Stiffness_[24:47]) correspond to stif[1][0:23], and so on.

4.2.2.9 get_StrainsMITC4()

```
vector< double > NonlinearShellMITC4::get_StrainsMITC4 ( )
```

Calculate and get the strain tensor (Strains_) at all Gauss points of the shell element. The strains are defined in the local coordinate system at the Gauss point.

Returns

vector<double> The strain input format is defined as follows:
Exx, Eyy, Exy, Exz, Eyz, i.e., five strain components per integration point. The strains are stored in the "↩ Strains_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.↩ 0), the ordering in Strains_ is:
[Exx_I^zet1, Eyy_I^zet1, Exy_I^zet1, Exz_I^zet1, Eyz_I^zet1,...,Exx_IV^zet1, Eyy_IV^zet1, Exy_IV^zet1, Exz_IV^zet1, Eyz_IV^zet1,
Exx_I^zet2, Eyy_I^zet2, Exy_I^zet2, Exz_I^zet2, Eyz_I^zet2,...,Exx_IV^zet2, Eyy_IV^zet2, Exy_IV^zet2, Exz_IV^zet2, Eyz_IV^zet2]

4.2.2.10 get_StrainsMITC4_Global()

```
vector< double > NonlinearShellMITC4::get_StrainsMITC4_Global ( )
```

Calculate and get the strain tensor (Strains_Global_) at all Gauss points of the shell element. The strains are defined in the global coordinate system.

Returns

vector<double> The strain input format is defined as follows:
EGxx, EGyy, EGxy, EGxz, EGyz, i.e., five strain components per integration point. The strains are stored in the "Strains_Global_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.↩ 0), the ordering in Strains_Global_ is:
[EGxx_I^zet1, EGyy_I^zet1, EGxy_I^zet1, EGxz_I^zet1, EGyz_I^zet1,...,EGxx_IV^zet1, EGyy_IV^zet1, EGxy_IV^zet1, EGxz_IV^zet1, EGyz_IV^zet1,
EGxx_I^zet2, EGyy_I^zet2, EGxy_I^zet2, EGxz_I^zet2, EGyz_I^zet2,...,EGxx_IV^zet2, EGyy_IV^zet2, EGxy_IV^zet2, EGxz_IV^zet2, EGyz_IV^zet2]

4.2.2.11 get_StressesMITC4()

```
vector< double > NonlinearShellMITC4::get_StressesMITC4 ( )
```

Calculate and get the stress tensor (stresses_) at all Gauss points of the shell element. The stresses are defined in the local coordinate system at the Gauss point.

Returns

vector<double> The stress input format is defined as follows:
Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.↩ 0), the ordering in stresses_ is:
[Sxx_I^zet1, Syy_I^zet1, Sxy_I^zet1, Sxz_I^zet1, Syz_I^zet1,...,Sxx_IV^zet1, Syy_IV^zet1, Sxy_IV^zet1, Sxz_IV^zet1, Syz_IV^zet1,
Sxx_I^zet2, Syy_I^zet2, Sxy_I^zet2, Sxz_I^zet2, Syz_I^zet2,...,Sxx_IV^zet2, Syy_IV^zet2, Sxy_IV^zet2, Sxz_IV^zet2, Syz_IV^zet2]

4.2.2.12 get_StressesMITC4_Global()

```
vector< double > NonlinearShellMITC4::get_StressesMITC4_Global ( )
```

Calculate and get the stress tensor (stresses_Global) at all Gauss points of the shell element. The stresses are defined in the global coordinate system.

Returns

vector<double> The stress input format is defined as follows:
SGxx, SGyy, SGxy, SGxz, SGyz, i.e., five stress components per integration point. The stresses are stored in the "stresses_Global" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.↩ 0), the ordering in stresses_ is:
[SGxx_I^zet1, SGyy_I^zet1, SGxy_I^zet1, SGxz_I^zet1, SGyz_I^zet1,...,SGxx_IV^zet1, SGyy_IV^zet1, SGxy_IV^zet1, SGxz_IV^zet1, SGyz_IV^zet1,
SGxx_I^zet2, SGyy_I^zet2, SGxy_I^zet2, SGxz_I^zet2, SGyz_I^zet2,...,SGxx_IV^zet2, SGyy_IV^zet2, SGxy_IV^zet2, SGxz_IV^zet2, SGyz_IV^zet2]

4.2.2.13 set_LocalAxes_All()

```
void NonlinearShellMITC4::set_LocalAxes_All (
            vector< double > axesgp_in )  [inline]
```

Set the local axes at the Gauss integration points of the element. If the local axes are not setted then the API will calculate the local axes at the GPs. If the MITC4 shell is to be used for composites or planar plastic anisotropy then the local axes at the GPs should be setted after the instantiation of the shell MITC4 class.

Parameters

axesgp_in	the structure for "axesgp_in" must be the following:
	Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) {
	Loop in the 4 in-plane GPs: for (int ig = 0; ig < 4; ++ig) {
	axesgp[0:2] = x-local vector
	axesgp[3:5] = y-local vector
	axesgp[6:8] = z-local vector
	for (int i = 0; i < axesgp.size(); ++i) { axesgp_in.push_back(axesgp[i]); } } }

The documentation for this class was generated from the following file:

- /home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearShellMITC4/NonlinearShellMITC4.h

Chapter 5

File Documentation

5.1 NonlinearShellMCYSE.h

```
00001
00002 #include <vector>
00003 #include <math.h>
00004 #include <iostream>
00005
00006 using namespace std;
00007
00008 //#define ACTIVATE_NonlinearShellMCYSE_LICENSE(pathToLicense)   NonlinearShellMCYSE::path =
      pathToLicense;
00009
00010 class NonlinearShellMCYSE {
00011 public:
00012
00013     // constructor
00056     NonlinearShellMCYSE(string path_in,
00057                 vector<vector<double» &xyz_in,
00058                 vector<vector<double» &vnor_in,
00059                 vector<double> &thick_in,
00060                 vector<vector<double» &xlocal_in,
00061                 vector<vector<double» &ylocal_in,
00062                 vector<double> &zetGPcoord_in,
00063                 vector<double> &zetGPweigth_in,
00064                 vector<double> &stresses_in,
00065                 vector<double> &DMatrix_in);
00066
00101     NonlinearShellMCYSE(string path_in,
00102                 vector<vector<double» &xyz_in,
00103                 vector<vector<double» &vnor_in,
00104                 vector<double> &thick_in,
00105                 vector<vector<double» &xlocal_in,
00106                 vector<vector<double» &ylocal_in,
00107                 vector<double> &zetGPcoord_in,
00108                 vector<double> &Displacements_in,
00109                 vector<double> &DMatrix_in);
00110
00160     NonlinearShellMCYSE(string path_in,
00161                  vector<vector<double» &xyz_in,
00162                  vector<vector<double» &vnor_in,
00163                  vector<double> &thick_in,
00164                  vector<vector<double» &xlocal_in,
00165                  vector<vector<double» &ylocal_in,
00166                  vector<double> &zetGPcoord_in,
00167                  vector<double> &zetGPweigth_in,
00168                  vector<double> &Displacements_in,
00169                  vector<double> &stresses_in,
00170                  vector<double> &DMatrix_in,
00171                  vector<double> &stresses_hourglass_in);
00172
00173
00174     inline static string path = "";
00175
00176     // setters & accessors
00177     //
00178     size_t get_Rows() { return sizeR_; }
00179     size_t get_Cols() { return sizeC_; }
00180
00181     // B-matrix in vector form (in the local csys)
00182     void get_BMatrix(vector<double> *BMat_out) { *BMat_out = BMat_; }
```

```
00183
00184     // local (csys) transposed B-matrix in vector form
00185     void get_BMatrix_Transposed(vector<double> *BMatT) { *BMatT = BMatTransposed_; }
00186
00187     // determinant of the Jacobian
00188     double get_determinant() { return determinant_; }
00189
00190     // get the linear Stiffness Matrix
00198     void Calculate_StiffnessMCYSE();
00208     vector<double> get_StiffnessMCYSE();
00209
00210
00211     // setter for local axes at the single in-plane GP at the centre of the element
00212     // in case the user decides to load the local axes (eg for plastic anisotropic csys)
00213     // the structure for "axesgp_" must be the following:
00214     // axesgp_in[0:2] = x-local vector
00215     // axesgp_in[3:5] = y-local vector
00216     // axesgp_in[6:8] = z-local vector
00217     //
00218     // Note: if local axes are not setted, the API will calculate them
00225     void set_LocalAxes(vector<double> axesgp_in) { axesgp_all_ = axesgp_in; }
00231     vector<double> get_LocalAxes_All() { return axesgp_all_; }
00232
00233     // calculates the local axes at each in-plane Gauss point. The local axes at the in-plane GP
00234     // are located at the mid-surface (zet = 0.0) of the shell so that for non-zero zet coordinate the
00235     // local axes are the same as at zet = 0.0.
00236     void Calculate_Local_Axes_All_GP();
00237
00238     // get the strain tensors
00250     vector<double> get_StrainsMCYSE();
00262     vector<double> get_StrainsMCYSE_Global();
00263
00264     //
00276     vector<double> get_StressesMCYSE();
00288     vector<double> get_StressesMCYSE_Global();
00289
00290     // rotate axes at the GPs
00291     void rotate_GP_axes();
00292
00293     // acessor to the Incremental hourglass force calculated at
00294     // the mid-configuration
00295     vector<double> Get_Force_Hourglass() { return Force_Hourglass_; }
00296     void Hourglass_Forces_Intermediate();
00297
00298     // Internal force vector for Explicit
00303     void Calculate_Internal_Forces_Explicit();
00311     vector<double> Get_Internal_Forces_Explicit() { return Force_Internal_; }
00312
00313     // getter for the hourglass stresses
00314     vector<double> Get_stresses_hourglass() { return stresses_hourglass_; }
00315
00316
00317 private:
00318
00319     int igaus_;
00320     double zet_;
00321     double zet_weight_;
00322     double determinant_; // determinant of the Jacobian
00323
00324     vector<vector<double» xyz_; // coordinates for the 4-nodes of the shell
00325     vector<vector<double» vnor_; // normal vector at each node
00326     vector<double> thick_; // thickness at each node
00327     vector<vector<double» xlocal_; // x-local nodal csys at each node
00328     vector<vector<double» ylocal_; // y-local nodal csys at each node
00329     vector<double> zetGPcoord_; // zet coordinate of GP
00330     vector<double> zetGPweigth_; // GP weight along zet
00331
00332     vector<double> axesgp_all_; // local axes at all in-plane GPs. The local axes are defined
00333                                 // for in-plane GPs only (zet = 0)
00334     vector<double> axesgp_; // local axes at the GP
00335     vector<double> axesgp_all_temp_;
00336
00337
00338     // "BMat_" is in vector form. It is calculated and stored row-wise as follows:
00339     //     BMat_[0:23]   = B[0,0:24]
00340     //     BMat_[24:47]  = B[1,0:24]
00341     //     BMat_[48:71]  = B[2,0:24]
00342     //     BMat_[72:95]  = B[3,0:24]
00343     //     BMat_[96:119] = B[4,0:24]
00344     vector<double> BMat_; // final B-matrix in vector form
00345     vector<double> BMatTransposed_; // transposed B-matrix
00346     size_t sizeR_; // number of rows of the B-matrix
00347     size_t sizeC_; // number of columns of the B-matrix
00348
00349     //
00350     // D-Matrix vector with the D-Matrix in vectorial form for all integration points.
00351     // each integration point has 25 (5x5) positions in the D-Matrix vector
```

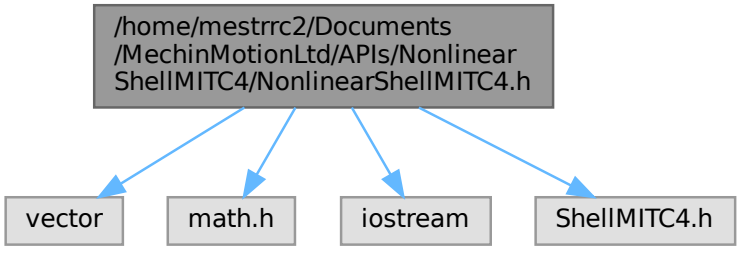
```
00352     vector<double> DMatrix_;
00353
00354     vector<vector<double» forma;
00355
00356     double qx,qy,qz,ex,ey,ez;
00357     double rhx,rhy,rhz,v3x,v3y,v3z;
00358     double qv3x,qv3y,qv3z;
00359     double ev3x,ev3y,ev3z;
00360     double rhv3x,rhv3y,rhv3z;
00361     double rja,rje,rji,rjm,rjq,rju,rjya,rjye,rjyi;
00362     double rjpa1,rjpa10,rjpa16;
00363
00364     double qsiixi;
00365     double qsiiyi;
00366     double etaixi;
00367     double etaiyi;
00368     double hjxj;
00369     double hjyj;
00370
00371     vector<double> gamai;
00372
00373     vector<vector<double» xcord;
00374
00375     vector<vector<double» rjac;
00376     vector<vector<double» raca;
00377     vector<vector<double» racb;
00378     vector<vector<double» racc;
00379     vector<vector<double» racd;
00380
00381     double rjaco1;
00382     double rjaco2;
00383     double rjaco3;
00384
00385
00386     vector<vector<double» rcl;
00387     vector<vector<double» rot;
00388
00389     vector<vector<double» rott;
00390
00391
00392     vector<vector<double» rlna;
00393
00394
00395     vector<double> b1a,b1b,b1c;
00396     vector<double> b2a,b2b,b2c;
00397     vector<double> b1aa,b2aa,b1bb,b2bb,b1ac,b2ac,b1bd,b2bd;
00398
00399
00400     vector<vector<double» stiffness; // linear Stiffness Matrix in matrix form
00401
00402     vector<double> Stiffness_; // linear Stiffness in vector form
00403
00404     vector<vector<double» tpp;
00405
00406
00407
00408     void StrainTransformMCYSE(vector<vector<double» &rocnat, vector<double> &ratsgn);
00409
00410     // cross-product
00411     vector<double> prodve(vector<double> vec1, vector<double> vec2, int khoice);
00412
00413     // determinant
00414     double determinant(vector<vector<double» &jacobian);
00415
00416     // norm of a vector
00417     double norm_vector(vector<double> v_in);
00418
00419     // calculates the local axes at the single in-plane Gauss point at the centre of the element.
00420     // The local axes are located at the mid-surface (zet = 0.0) of the shell so that for
00421     // non-zero zet coordinate the local axes are the same as at zet = 0.0.
00422     void Calculate_Local_Axes_GP();
00423
00424     // B-matrix in the convective csys
00425     //void BCovMITC4(int ig, double zet, vector<vector<double» &jacob, vector<double> &bcovar);
00426
00427     // method to be called for the calculation of the Linear Stiffness
00428     void StiffnessMCYSE();
00429
00430     // stabilisation (hourglass) stiffness matrix
00431     void StiffnessMCYSE_Hourglass();
00432
00433     // displacements vector
00434     vector<double> Displacements_;
00435
00436     // Strains
00437     vector<double> Strains_;
00438     vector<double> Strains_Global_;
```

```
00439
00440     // Strains
00441     void StrainsMCYSE();
00442
00443     // Stresses
00444     //
00445     // the stresses at the integration points must be given as input during class constructor
00446     // the stresses are required for the internal force vector and initial stress stiffness matrix
      calculations
00447     // the format of the stresses input is the following:
00448     // Sxx, Syy, Sxy, Sxz, Syz, that is, 5 stress components per integration point along the thickness
      direction
00449     // the order of the integration points in the "stresses_" vector is from the bottom-up direction,
00450     // from zet -1.0 to +1.0. For example, for 2 integration points along thickness, zet1 =
      -1.0/sqrt(3.0)
00451     // and zet2 = +1.0/sqrt(3.0), we get: stresses_[Sxx_zet1, Syy_zet1, Sxy_zet1, Sxz_zet1, Syz_zet1,
00452     // Sxx_zet2, Syy_zet2, Sxy_zet2, Sxz_zet2, Syz_zet2]
00453     vector<double> stresses_; // stresses at the integration points
00454     vector<double> stresses_Global; // stresses at the integration point at the Global csys
00455
00456     void StressesMCYSE();
00457
00458     void Strains_Transform();
00459
00460     void Stresses_Transform();
00461
00462     vector<double> Mat_Transpose(size_t sizeA_R, size_t sizeA_C, vector<double> A);
00463
00464     vector<double> Mat_Mult(size_t sizeR, size_t sizeC, size_t sizeCommon, vector<double> B,
      vector<double> C);
00465
00466     //
00467     vector<vector<double» rnstrain(vector<vector<double» fk);
00468
00469     // Internal Forces
00470     vector<double> Force_Hourglass_; // hourglass
00471     vector<double> Force_Internal_;
00472
00473     // hourglass stresses
00474     vector<double> stresses_hourglass_;
00475
00476
00477 };
00478
00479
```

5.2 /home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearShell↩ MITC4/NonlinearShellMITC4.h File Reference

```
#include <vector>
#include <math.h>
#include <iostream>
#include "ShellMITC4.h"
```

Include dependency graph for NonlinearShellMITC4.h:



Classes

- class NonlinearShellMITC4

5.2.1 Detailed Description

Author

Rui Cardoso (support@mechinmotion.com)

Version

1.0

Date

2025-12-30

Copyright

5.3 NonlinearShellMITC4.h

Go to the documentation of this file.

```
00001
00011 #include <vector>
00012 #include <math.h>
00013 #include <iostream>
00014 #include "ShellMITC4.h"
00015
00016 using namespace std;
00017
00018 //#define ACTIVATE_NonlinearShellMITC4_LICENSE(pathToLicense)   NonlinearShellMITC4::path =
      pathToLicense;
00019
00020 class NonlinearShellMITC4 {
00021 public:
00022
00023     // constructor
00068     NonlinearShellMITC4(string path_in, vector<vector<double» &xyz_in,
00069                 vector<vector<double» &vnor_in,
00070                 vector<double> &thick_in,
00071                 vector<vector<double» &xlocal_in,
00072                 vector<vector<double» &ylocal_in,
00073                 vector<double> &zetGPcoord_in,
00074                 vector<double> &zetGPweigth_in,
00075                 vector<double> &stresses_in,
00076                 vector<double> &DMatrix_in);
00077
00078     // constructor
00114     NonlinearShellMITC4(string path_in, vector<vector<double» &xyz_in,
00115                 vector<vector<double» &vnor_in,
00116                 vector<double> &thick_in,
00117                 vector<vector<double» &xlocal_in,
00118                 vector<vector<double» &ylocal_in,
00119                 vector<double> &zetGPcoord_in,
00120                 vector<double> &Displacements_in,
00121                 vector<double> &DMatrix_in);
00122
00123
00124     inline static string path = "";
00125
00126     // setters & accessors
00127     //
00128     size_t get_Rows() { return sizeR_; }
```

```
00129     size_t get_Cols() { return sizeC_; }
00130
00131     // B-matrix in vector form (in the local csys)
00146     void get_BMatrix(int igaus_in, int izet_in, double zet_in, vector<double> *BMat_out);
00147
00148     // local (csys) transposed B-matrix in vector form
00149     // warning: "get_BMatrix" must be called first to construct the B-Matrix
00150     // only then the transposed B-Matrix can be called
00158     void get_BMatrix_Transposed(vector<double> *BMatT) { *BMatT = BMatTransposed_; }
00159
00160     // determinant of the Jacobian
00161     //double get_determinant() { return determinant_; }
00162
00163     // setter for local axes at in-plane GP in case the user decides to load
00164     // the local axes at the GP (eg for plastic anisotropic csys)
00165     // the structure for "axesgp_" must be the following:
00166     // axesgp_in[0:2] = x-local vector
00167     // axesgp_in[3:5] = y-local vector
00168     // axesgp_in[6:8] = z-local vector
00169     //
00170     // Note: if local axes are not setted, the API will calculate them
00185     void set_LocalAxes_All(vector<double> axesgp_in) { axesgp_all_ = axesgp_in; }
00193     vector<double> get_LocalAxes_All() { return axesgp_all_; }
00194
00195     // calculates the local axes at each in-plane Gauss point. The local axes at the in-plane GP
00196     // are located at the mid-surface (zet = 0.0) of the shell so that for non-zero zet coordinate the
00197     // local axes are the same as at zet = 0.0.
00203     void Calculate_Local_Axes_All_GP();
00204
00205     // get the linear Stiffness Matrix
00216     vector<double> Calculate_StiffnessMITC4();
00224     vector<double> get_StiffnessMITC4() { return Stiffness_; }
00225
00226     // get the initial stress (geometric) Stiffness Matrix
00236     vector<double> get_Stiffness_InitialStress();
00237
00238     // get the Internal Forces vector
00245     vector<double> Calculate_Internal_Forces();
00251     vector<double> get_Internal_Forces() { return Internal_Force_; }
00252
00253     // calculates the Stiffness and Internal Force vector in one go
00264     void Calculate_Stiffness_Internal();
00265
00266     // get the strain tensors
00278     vector<double> get_StrainsMITC4();
00290     vector<double> get_StrainsMITC4_Global();
00291
00292     // rotate axes at the GPs
00293     void rotate_GP_axes();
00294
00295     // get the stress tensors
00307     vector<double> get_StressesMITC4();
00319     vector<double> get_StressesMITC4_Global();
00320
00321     //
00322     void Jacobi(vector<vector<double» &ain, vector<vector<double» &bin,
00323                 vector<double> &eigenvalues_, vector<vector<double» &eigenvectors_);
00324
00325
00326
00327 private:
00328
00329     //void Permitelemento(string path);
00330
00331     int igaus_;
00332
00333     int izet_;
00334
00335     double zet_;
00336     double zet_weight_;
00337     double determinant_; // determinant of the Jacobian
00338
00339     vector<vector<double» xyz_; // coordinates for the 4-nodes of the shell
00340     vector<vector<double» vnor_; // normal vector at each node
00341     vector<double> thick_; // thickness at each node
00342     vector<vector<double» xlocal_; // x-local nodal csys at each node
00343     vector<vector<double» ylocal_; // y-local nodal csys at each node
00344     vector<double> qsiGPcoord_; // qsi coordinate of GP
00345     vector<double> etaGPcoord_; // eta coordinate of GP
00346     vector<double> zetGPcoord_; // zet coordinate of GP
00347     vector<double> qsiGPweigth_; // GP weight along qsi
00348     vector<double> etaGPweigth_; // GP weight along eta
00349     vector<double> zetGPweigth_; // GP weight along zet
00350     vector<vector<double» fformaGP_; // shape functions at each GP
00351     vector<vector<double» fforma_;
00352     vector<vector<double» qformaGP_; // qsi shape functions derivatives at each GP
00353     vector<vector<double» qforma_;
```

```
00354     vector<vector<double» eformaGP_; // eta shape functions derivatives at each GP
00355     vector<vector<double» eforma_;
00356     vector<vector<double» zformaGP_; // zet shape functions derivatives at each GP
00357     vector<double> axesgp_all_; // local axes at all in-plane GPs. The local axes are defined
00358                                 // for in-plane GPs only (zet = 0)
00359     vector<double> axesgp_; // local axes at the GP
00360     vector<double> axesgp_all_temp_;
00361
00362     //
00363     vector<double> Displacements_;
00364
00365     //
00366     vector<double> Strains_;
00367     vector<double> Strains_Global_;
00368
00369     //
00370     // the stresses at the integration points must be given as input during class constructor
00371     // the stresses are required for the internal force vector and initial stress stiffness matrix
      calculations
00372     // the format of the stresses input is the following:
00373     // Sxx, Syy, Sxy, Sxz, Syz, that is, 5 stress components per integration point
00374     // the order of the integration points in the "stresses_" vector is from the bottom-up direction,
00375     // from zet -1.0 to +1.0. For example, for 2 integration points along thickness, zet1 =
      -1.0/sqrt(3.0)
00376     // and zet2 = +1.0/sqrt(3.0), we get: stresses_[Sxx_I^zet1, Syy_I^zet1, Sxy_I^zet1, Sxz_I^zet1,
      Syz_I^zet1,
00377     // ..., Sxx_IV^zet1, Syy_IV^zet1, Sxy_IV^zet1, Sxz_IV^zet1, Syz_IV^zet1,Sxx_I^zet2, Syy_I^zet2,
      Sxy_I^zet2,
00378     // Sxz_I^zet2, Syz_I^zet2,..., Sxx_IV^zet2, Syy_IV^zet2, Sxy_IV^zet2, Sxz_IV^zet2, Syz_IV^zet2]
00379     vector<double> stresses_; // stresses at the integration points
00380     vector<double> stresses_Global; // stresses at the integration point at the Global csys
00381
00382     //
00383     // D-Matrix vector with the D-Matrix in vectorial form for all integration points.
00384     // each integration point has 25 (5x5) positions in the D-Matrix vector
00385     vector<double> DMatrix_;
00386
00387
00388     // "BMat_" is in vector form. It is calculated and stored row-wise as follows:
00389     //     BMat_[0:23]   = B[0,0:24]
00390     //     BMat_[24:47]  = B[1,0:24]
00391     //     BMat_[48:71]  = B[2,0:24]
00392     //     BMat_[72:95]  = B[3,0:24]
00393     //     BMat_[96:119] = B[4,0:24]
00394     vector<double> BMat_; // final B-matrix in vector form at the integration point
00395     vector<double> BMatTransposed_; // transposed B-matrix at the integration point
00396
00397     //
00398     size_t sizeR_; // number of rows of the B-matrix
00399     size_t sizeC_; // number of columns of the B-matrix
00400
00401     // linear Stiffness Matrix
00402     vector<double> Stiffness_; // linear Stiffness Matrix
00403     vector<double> Stiffness_Gauss_;
00404
00405     // initial stress stiffness matrix
00406     vector<double> StiffnessInitialStress_; // initial stress stiffness matrix
00407
00408     // Internal Force Vector
00409     vector<double> Internal_Force_; // Internal Force Vector
00410     vector<double> Internal_Force_Gauss_;
00411
00412     // initialise matrix in vector form
00413     void initialise_matrix_inVector(size_t lines, size_t columns, vector<double> &mat);
00414
00415     // cross-product
00416     vector<double> prodve(vector<double> vec1, vector<double> vec2, int khoice);
00417
00418     // determinant
00419     double determinant(vector<vector<double» &jacobian);
00420
00421     // 3x3 inverse
00422     void inverse_3x3(vector<vector<double» &racobb, double determ, vector<vector<double» &racobi);
00423
00424     // norm of a vector
00425     double norm_vector(vector<double> v_in);
00426
00427     // initialisation of a matrix
00428     void initialise_matrix(int lines, int columns, vector<vector<double»& mat);
00429
00430     // calculates the Jacobian (3x3) matrix at each GP defined by the in-plane GP number "igau"
00431     // and parametric coordinate along thickness direction "zet"
00432     vector<vector<double» Jacobian(int igau, double zet);
00433
00434     // Matrix multiplication
00435     vector<double> Mat_Mult(size_t sizeR, size_t sizeC, size_t sizeCommon, vector<double> B,
      vector<double> C);
```

```
00436
00437     vector<double> Mat_Mult_Upper_Triangle(size_t sizeR, size_t sizeC, size_t sizeCommon,
      vector<double> B, vector<double> C);
00438
00439     // transpose a matrix in vector form
00440     vector<double> Mat_Transpose(size_t sizeA_R, size_t sizeA_C, vector<double> A);
00441
00442     // load shape functions and derivatives
00443     void Load_Shape_Derivatives(int shift);
00444
00445     // Linear Stiffness Matrix
00446     void Stiffness_MITC4();
00447
00448     // stiffness matrix at a Gauss Point
00449     vector<double> Stiffness_Gauss_MITC4(vector<double> DMatrix_in);
00450
00451     // stiffness matrix and internal force vector at a Gauss Point
00452     void Stiffness_Internal_Gauss_MITC4(vector<double> DMatrix_in, vector<double> Stresses_in);
00453
00454     // initial stress stiffness matrix
00455     void calculate_StiffnessInitialStress();
00456
00457     // duvw matrix for the initial stress-stiffness matrix
00458     vector<vector<double» duvw_Gauss_point(vector<vector<double» racobi);
00459
00460     // Internal Force vector
00461     void calculate_Internal_Forces();
00462
00463     //
00464     void StrainsMITC4();
00465     void Strains_Transform();
00466
00467     //
00468     vector<vector<double» rnstrain(vector<vector<double» fk);
00469
00470     void StressesMITC4();
00471     void Stresses_Transform();
00472
00473 };
00474
00475
```